

Foundations of Quantum Computing

– Week 5 – Explore – Study

Niel De Beaudrap

5 Grover’s algorithm and amplitude amplification

One of the things for which quantum computation is somewhat famous, is its ability to ‘search a database in parallel’. This description is an oversimplification of something that a quantum computer actually can do, which is a technique known as ‘amplitude amplification’. This is a procedure to take a uniform superposition of computational basis states $|x\rangle$ and increase the amplitude for those values of x which satisfy some computable property. This week, we will consider how this is done.

In this week’s Study section you will cover the following topics:

- 5.1 How ‘search’ problems can be described.
- 5.2 Grover’s search algorithm, to search for a unique element.
- 5.3 Adapting Grover’s algorithm, to search without uniqueness.

5.1 How ‘search’ problems can be described

The problem that we are looking to solve is often described as a ‘search’ problem. This is technically correct, but we should understand that it isn’t the sort of ‘searching’ that you do for a record in a database: it’s the sort of ‘searching’ that you do for a missing dog, in a forest, at night, without a torch.

We consider a situation where we are presented an oracle $\mathbf{0_f} : \{0, 1\}^n \times \{0, 1\} \rightarrow \{0, 1\}^n \times \{0, 1\}$ which computes:

$$\mathbf{0_f}(x, t) = (x, t \oplus f(x)) \tag{5.1}$$

for some function $f : \{0, 1\}^n \rightarrow \{0, 1\}$. We are not provided with any information about how $\mathbf{0_f}$ is realised, or any other structure that the function f might have, so the only thing (classically) that we can do with this oracle is to query it for particular values of $x \in \{0, 1\}^n$ to see what the value of $f(x)$ is. The problem that we are interested in solving is then:

The search problem.

Given such an oracle $\mathbf{0_f}$, determine if there is a value of $x \in \{0, 1\}^n$ such that $f(x) = 1$, and produce such a value of x if so.

This is generally a difficult problem to solve quickly. We can of course solve the problem by just trying each possible input $x \in \{0, 1\}^n$ in turn until we find an input such that $f(x) = 1$: but there are 2^n such input strings, and we would end up trying all of them in

the case that $f(x) = 0$ for all x . We know nothing of how `0.f` is implemented, so the best that we can do is to systematically check all of the inputs (though perhaps by checking random values, to avoid the possibility of being unlucky in starting point for a sequential search). In fact, it is possible to show that no matter what approach one takes to the ‘oracle’ version, we expect to have to evaluate f using the oracle `0.f` about 2^{n-1} times, and in the worst case will have to evaluate it 2^n times.

Note that, even if we did know exactly how f were computed (in other words: if `0.f` were not an ‘oracle’, but instead a subroutine with a known implementation), and furthermore if f were efficiently computable, this problem would amount to the ‘Boolean satisfiability’ problem **SAT** – and it is expected that there is no efficient way to solve this problem. Anyone who can prove otherwise stands to become quite famous and possibly rich, as being able to solve **SAT** would also make many practically important optimisation problems much easier to solve. If there was a way to solve this problem **efficiently** by any means, it would likely radically affect the way that we organise society.

Unfortunately, there is no efficient quantum algorithm for the search problem. What we can do, is show that the best possible quantum algorithm for the search problem is much faster than $\mathcal{O}(2^n)$ time.

5.2 Grover’s search algorithm

We first consider the special case, where $f(a) = 1$ for **exactly one** bitstring $a = a_1a_2 \dots a_n \in \{0,1\}^n$. This is the setting for one of the most famous quantum algorithms: Grover’s algorithm. In this case, provided a unitary version of the oracle `0.f` corresponding to the classical reversible version, it is possible to find that specific bitstring a , in $\mathcal{O}(2^{n/2}) \subseteq \mathcal{O}(1.4143^n)$ evaluations of the oracle `0.f`. This is still an exponential number of queries as n increases. But it is an exponential which scales much more slowly than the classical requirement of 2^n evaluations of `0.f`.

5.2.1 Reducing the problem to two dimensions

Grover’s algorithm starts, as many quantum algorithms do, by preparing a register ‘ q ’ of n in the uniform superposition state:

$$|u\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle = \frac{1}{\sqrt{2^n}} |00 \dots 00\rangle + \frac{1}{\sqrt{2^n}} |00 \dots 01\rangle + \dots + \frac{1}{\sqrt{2^n}} |11 \dots 11\rangle . \quad (5.2)$$

A single one of the computational basis states $|a\rangle$ in this superposition satisfies $f(a) = 1$. The distinction between the other terms is not important – so, we might as well consider them all together as a single unit vector $|\beta\rangle$, which is a superposition of states $|x\rangle$ such that $f(x) = 0$:

$$|\beta\rangle = \frac{1}{\sqrt{2^n - 1}} \sum_{\substack{x \in \{0,1\}^n \\ x \neq a}} |x\rangle = \frac{1}{\sqrt{2^n - 1}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=0}} |x\rangle . \quad (5.3)$$

Figure 5.1: The 2D plane containing $|a\rangle$ and the uniform superposition. The uniform superposition can be decomposed as a linear combination of $|a\rangle$, and a state-vector $|\beta\rangle$ which is itself the uniform superposition of all states $|x\rangle$ such that $f(x) = 0$. We may illustrate the relationships between $|a\rangle$, $|\beta\rangle$ and $|u\rangle$ by a 2D picture as shown here, indicating the (small) angle between $|u\rangle$ and $|\beta\rangle$. Note that the angle θ satisfies $\sin(\theta) = 1/\sqrt{2^n}$ by construction.

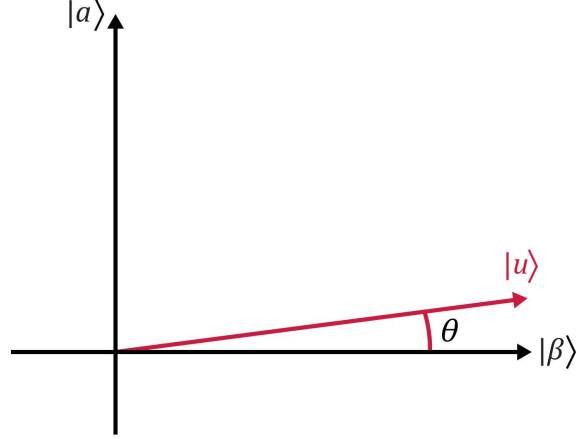


Figure 5.1 Long description: Two axes are marked in black, with $|a\rangle$ on the y-axis and $|\beta\rangle$ the x-axis. The vector $|u\rangle$ is indicated in dark red, starting where the axes meet and pointing right and slightly above the horizontal. The angle this vector makes with the horizontal axis is marked as θ .

The leading scalar factor in this case is $1/\sqrt{2^n - 1}$, as there are $2^n - 1$ different terms, so that $\langle\beta|\beta\rangle = 1$. Note that $|a\rangle$ and $|\beta\rangle$ are orthogonal, as they have no components in common in the computational basis. Then, we can write $|u\rangle$ as a superposition of the two states:

$$|u\rangle = \frac{1}{\sqrt{2^n}} |a\rangle + \frac{1}{\sqrt{2^n}} \sum_{\substack{x \in \{0,1\}^n \\ x \neq a}} |x\rangle = \frac{1}{\sqrt{2^n}} |a\rangle + \frac{\sqrt{2^n-1}}{\sqrt{2^n}} |\beta\rangle = \frac{1}{\sqrt{2^n}} |a\rangle + \sqrt{1 - \frac{1}{2^n}} |\beta\rangle. \quad (5.4)$$

We can sketch this as a diagram in the 2D plane, as the amplitudes of both $|a\rangle$ and $|\beta\rangle$ are real-valued. This is shown in Figure 5.1. (Bear in mind that we don't **know** what $|a\rangle$ is; we only know that it exists.)

For $n > 1$, the amplitude $\langle\beta|u\rangle = \sqrt{1 - 1/2^n}$ will be larger than the amplitude $\langle a|u\rangle$: this means that $|u\rangle$ is more closely aligned with $|\beta\rangle$, so the angle θ between them is smaller: we choose this angle as the basis for how we will analyse this problem. (The angle θ between $|u\rangle$ and $|\beta\rangle$ will depend with n : the illustration in Figure 5.1 is not intended to represent the actual angle formed for any particular value of n .) By construction, the particular value of θ is the unique angle between 0 and $\frac{\pi}{2}$ such that $\sin(\theta) = 1/\sqrt{2^n}$. In fact, we can approximate $\theta \approx 1/\sqrt{2^n}$ to a little more than n bits of precision for any $n > 0$.

Note that very few of the states that the register ‘ $\mathbf{q}$ ’ can be in, are vectors in this plane. (This is only a 2D plane after all, and the full set of states of $\mathbf{q}$ are the unit-vectors of the 2^n -dimensional vector space $\mathbb{C}^{2^n}$.) However, it would be convenient to try to describe

how to bring $\mathbf{q}$ into the state $\mathbf{a}$, or some state close to it, by considering just the vectors in this plane. This is what we will do next.

5.2.2 Operations which preserve the $|a\rangle, |\beta\rangle$ plane

Consider what operations we are able to perform, relating to the states illustrated in Figure 5.1, which preserve the 2D plane shown there. As this plane of state-vectors is not preserved by very many operations that we can do on $\mathbf{q}$, we must consider what operations **do** preserve this plane.

To be specific: suppose that the register ‘ $\mathbf{q}$ ’ is in some state $|\psi\rangle = c_0 |a\rangle + c_1 |\beta\rangle$ for some $c_0, c_1 \in \mathbb{C}$. The operations which we know that we can perform which relate to these states, are as follows:

- With an additional qubit $\mathbf{t}$, we can distinguish $|a\rangle$ from $|\beta\rangle$ using the oracle `0_f`: for instance, we have:

$$|\psi, 0\rangle_{\mathbf{q}, \mathbf{t}} = c_0 |a, 0\rangle_{\mathbf{q}, \mathbf{t}} + c_1 |\beta, 0\rangle_{\mathbf{q}, \mathbf{t}} \xrightarrow{0_f} c_0 |a, 1\rangle_{\mathbf{q}, \mathbf{t}} + c_1 |\beta, 0\rangle_{\mathbf{q}, \mathbf{t}}. \quad (5.5)$$

This strictly speaking does not preserve the plane, as afterwards $\mathbf{q}$ will generally be entangled with $\mathbf{t}$, but it will be a superposition of terms in which $\mathbf{q}$ is in some state described by a vector in the $|a\rangle, |\beta\rangle$ plane.

- We can prepare the state $|u\rangle$ from the state $|00 \cdots 0\rangle$ and also the reverse.

We must consider how we may use these operations to describe transformations of the plane.

(a) Reflecting about the $|\beta\rangle$ axis using phase kicks

One thing that we might do (simply because we can), is consider how `0_f` might be used to produce a phase-kick. If we consider an additional qubit $\mathbf{t}$ in the state $|-\rangle$, the effect of `0_f` will then be:

$$\begin{aligned} |\psi\rangle_{\mathbf{q}} \otimes |-\rangle_{\mathbf{t}} &= \frac{c_0}{\sqrt{2}} |a, 0\rangle_{\mathbf{q}, \mathbf{t}} - \frac{c_0}{\sqrt{2}} |a, 1\rangle_{\mathbf{q}, \mathbf{t}} + \frac{c_1}{\sqrt{2}} |\beta, 0\rangle_{\mathbf{q}, \mathbf{t}} - \frac{c_1}{\sqrt{2}} |\beta, 1\rangle_{\mathbf{q}, \mathbf{t}} \\ &\xrightarrow{0_f} \frac{c_0}{\sqrt{2}} |a, 1\rangle_{\mathbf{q}, \mathbf{t}} - \frac{c_0}{\sqrt{2}} |a, 0\rangle_{\mathbf{q}, \mathbf{t}} + \frac{c_1}{\sqrt{2}} |\beta, 0\rangle_{\mathbf{q}, \mathbf{t}} - \frac{c_1}{\sqrt{2}} |\beta, 1\rangle_{\mathbf{q}, \mathbf{t}} \\ &= c_0 |a\rangle_{\mathbf{q}} \otimes \frac{1}{\sqrt{2}} (|1\rangle - |0\rangle)_{\mathbf{t}} + c_1 |\beta\rangle_{\mathbf{q}} \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)_{\mathbf{t}} \\ &= -c_0 |a\rangle_{\mathbf{q}} \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)_{\mathbf{t}} + c_1 |\beta\rangle_{\mathbf{q}} \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)_{\mathbf{t}} \\ &= (-c_0 |a\rangle + c_1 |\beta\rangle)_{\mathbf{q}} \otimes |-\rangle_{\mathbf{t}}. \end{aligned} \quad (5.6)$$

At the end, $\mathbf{q}$ is still in a state in the plane: the effect is to negate the $|a\rangle$ component and leave the $|\beta\rangle$ component unaffected. We may encapsulate this in a subroutine `R_f`, which we present in Figure 5.2.

Figure 5.2: Realisation in QAL of the reflection R_f . A simple procedure to realise the reflection R_f , which in the $|a\rangle, |\beta\rangle$ plane realises a reflection through the $|\beta\rangle$ axis. We do this using a phase-kick with an auxiliary qubit t in the state $|-\rangle$.

```

define R_f q :
  # realise a reflection about  $|\beta\rangle$ , in the  $|a\rangle, |\beta\rangle$  plane
  operand register q:  $n$  qubits

  qubit t      # prepare an auxiliary qubit  $t$ 
  MxInit t     # in the state  $|+\rangle$ , and then
  Z t          # flip it to the state  $|-\rangle$ 

  O_f q, t     # realise phase-kick
  stop

```

Figure 5.3: The effect of R_f on $|u\rangle$. The effect of reflecting the state $|u\rangle$ through the $|\beta\rangle$ axis. (Note that while the resulting vector is further away from the vector $|a\rangle$; in fact the magnitude of the inner product remains the same: $|\langle a | R_f | u \rangle|^2 = |\langle a | u \rangle|^2$.)

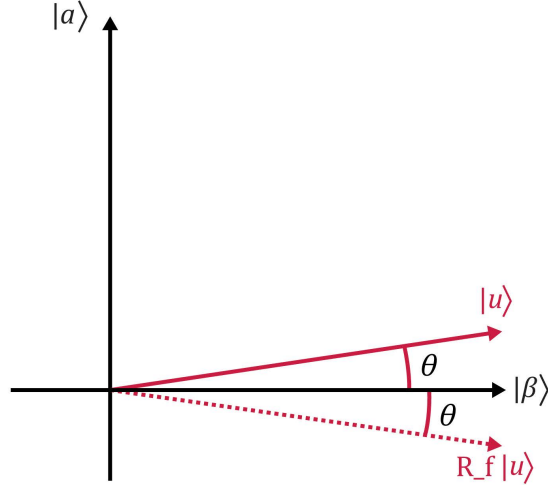


Figure 5.3 Long description: Two axes are marked in black, with $|a\rangle$ on the y-axis and $|\beta\rangle$ the x-axis. The vector $|u\rangle$ is indicated in dark red, starting where the axes cross and pointing right and slightly above the horizontal. The vector $R_f |u\rangle$ is indicated in dotted light red, pointing right and slightly below the horizontal. The angle these vectors make with the horizontal axis are both marked as θ .

For the 2D plane spanned by $|a\rangle$ and $|\beta\rangle$, the effect of R_f is a reflection through the $|\beta\rangle$ axis. We demonstrate this for an input state of $|u\rangle$ in particular, in Figure 5.3. Because $|u\rangle$ is at an angle from $|\beta\rangle$, the same is true of $R_f |u\rangle$ – so overall, this state is no closer to $|\beta\rangle$ than before. (It is further from the vector $|a\rangle$ than before, but it is closer to $-|a\rangle$, which represents the same state, so there is no observable change in this respect.) So while this operation preserves the plane and performs a non-trivial operation on it, it does not represent an operation which can help to solve the search problem on its own.

(b) Reflecting through the state $|u\rangle$ via multiply-controlled- Z operations

The other operation that we have access to is the ability to prepare $|u\rangle$ from the computational basis state $|00\cdots 0\rangle$ and by implication the ability to map $|u\rangle$ to the state $|00\cdots 0\rangle$. We may do either of these operations on the quantum register ‘ $\mathbf{q}$ ’, by performing a Hadamard operation on each qubit of $\mathbf{q}$.

Note that none of the other states in the $|a\rangle, |\beta\rangle$ plane will be transformed to a computational basis state by performing Hadamard operations on each qubit of $\mathbf{q}$. The only benefit to performing an operation which maps $|u\rangle \mapsto |00\cdots 0\rangle$ is if it can be used to help to perform an operation which singles out the state $|u\rangle$ in the $|a\rangle, |\beta\rangle$ plane.

Having covered a reflection about the $|\beta\rangle$ axis, we may consider what the effect would be of a reflection about the $|u\rangle$ state itself. We could realise this by a linear transformation:

$$\mathbf{R}_u = 2|u\rangle\langle u| - I. \quad (5.7)$$

To see that $\mathbf{R}_u$ is a reflection about $|u\rangle$, notice that:

$$(2|u\rangle\langle u| - I)|u\rangle = 2|u\rangle\langle u|u\rangle - |u\rangle = |u\rangle, \quad (5.8)$$

so that $|u\rangle$ is a +1-eigenvector. For any other vector $|w\rangle$ which is orthogonal to $|u\rangle$, we have instead:

$$(2|u\rangle\langle u| - I)|w\rangle = 2|u\rangle\langle u|w\rangle - |w\rangle = \mathbf{0} - |w\rangle = -|w\rangle, \quad (5.9)$$

so that such a vector $|w\rangle$ is a -1 -eigenvector. In particular, in the $|a\rangle, |\beta\rangle$ plane, this operation realises a reflection through the axis defined by $|u\rangle$. This is not a useful thing to apply to the state $|u\rangle$ itself, but it does have a useful effect on the state $\mathbf{R}_f|u\rangle$: this is illustrated in Figure 5.4.

The way we would realise the operation $\mathbf{R}_u$ would be to use Hadamard gates to reduce this to the problem of realising a reflection $R_0 = 2|00\cdots 0\rangle\langle 00\cdots 0| - I$, which leaves the state $|00\cdots 0\rangle$ unaffected but induces a change of sign on any other computational basis state $|x\rangle \neq |00\cdots 0\rangle$. In fact, we will describe a way to realise the operation $-R_0$: the change in sign makes no difference to the states or transformation being described.

To do this, we need to find a way to recognise and selectively apply a phase to the computational basis state $|00\cdots 0\rangle$ on n qubits, apart from the fact that we need to do this coherently, but this is essentially problem of classical computation. We can do this by writing a procedure which recognises the computational basis state $|11\cdots 1\rangle$, and apply this to an input where we have performed an X operation on all of the bits. A procedure to recognise the state $|11\cdots 1\rangle$, is essentially an implementation of an n -bit AND gate. Reversibly, we would do this by generalising the definition of `P_AND2`, `P_AND3`, and so forth to more and more bits. ‘Quantumly’, we do the same thing using `CCX` operations, using a recursively defined subroutine that performs the following (coherently) on a computational basis state:

Figure 5.4: The effect of reflecting the state $R_f |u\rangle$ through the $|u\rangle$ axis in the $|a\rangle, |\beta\rangle$ plane.

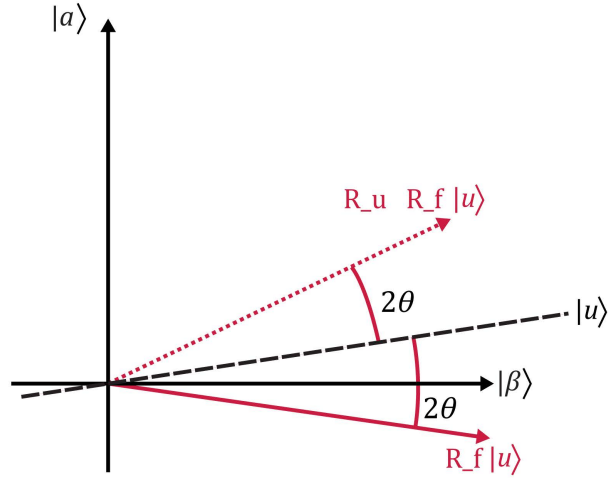


Figure 5.4 long description: Two axes are marked in black, with $|a\rangle$ on the y-axis and $|\beta\rangle$ the x-axis. A third axis defined by the state $|u\rangle$, is indicated by a black dashed line pointing to the right and slightly upwards and labelled by $|u\rangle$ on the right. The vector $R_f |u\rangle$ is indicated in dark red, starting where the axes intersect and pointing right and slightly below the horizontal. The angle this vector makes with the $|u\rangle$ axis is marked as 2θ . Another vector labelled by $R_u R_f |u\rangle$ is indicated in dotted light red, pointing to the right and with a significant vertical component. The angle this vector makes with the $|u\rangle$ axis is also marked as 2θ .

1. Compute whether the first $n - 1$ qubits $q[0], q[1], \dots, q[n-2]$ are in the state $|1\rangle$, and flip an auxiliary qubit $a[n-1]$ from $|0\rangle$ to $|1\rangle$ if so.
2. If the auxiliary qubit and $a[n-1]$ and the n^{th} input qubit $q[n-1]$ are both $|1\rangle$, apply X to the target qubit.
3. Uncompute the partial result stored in the auxiliary qubit $a[n-1]$.

A procedure defined along these lines is shown in Figure 5.5. Using such an operation, we can implement a procedure **AuxR_0** to realise the reflection R_0 defined in the preceding, making use of n qubits of workspace initialised to $|00 \dots 0\rangle$ to do so. (Strictly speaking, this operation could be thought of as realising the linear transformation $-R_0$, but the extra factor of -1 makes no difference to the transformations of the states themselves.) We describe an implementation of **AuxR_0** in Figure 5.6.

(c) The ‘Grover iterator’

We have defined the operations R_f and R_u which preserve the $|a\rangle, |\beta\rangle$ -plane. Furthermore, Figure 5.4 shows something interesting about these operations: the state $R_u R_f |u\rangle$ is closer to $|a\rangle$ than the state $|u\rangle$ is – specifically, by a rotation of 2θ .

It is possible to show that in fact the operation $G_f = R_u R_f$ is a rotation by 2θ , in the direction from the vector $|\beta\rangle$ to the vector $|a\rangle$. (We have seen how G_f affects the state $|u\rangle$; to see that it is a rotation of all vectors in this plane, it is enough to see that G_f is

Figure 5.5: A subroutine to compute multiply-controlled X operations. A recursive procedure to perform coherently-controlled X operations, with a large number of controls. The number of controls is indicated by an integer meta-parameter, n . This meta-parameter determines the size of a register q which is used as the controls of the operation. Due to the limitations on auxiliary memory in subroutines with meta-parameters, the additional workspace of $n-2$ qubits required by the subroutine must also be taken as an operand a . If $n = 2$, we don't make use of any auxiliary qubits and instead perform a CCX operation.

```

define MultiCtrlX[n]  q, a, t :
    # perform a multiply-controlled X operations, using:
    metaparameter integer n          # determines number of controls
    operand register q: n qubits      # register of control qubits
    operand register a: n-2 qubits    # register of 'auxiliary' qubits
    operand qubit t                   # the target qubit

    !jump_if_geq n, 3, @recursive_call
        CCX q[0], q[1], t
        !jump @done
    @recursive_call:
        MultiCtrlX[n-1]  q[0:n-2], a[0:n-4], a[n-3]
        CCX  q[n-1], a[n-3], t
        MultiCtrlX[n-1]  q[0:n-2], a[0:n-4], a[n-3]
    @done:
    stop

```

Figure 5.6: Realisation of the reflection R_u . We realise R_u by mapping $|u\rangle$ to $|11\cdots 1\rangle$, using a multiply-controlled- X operation to realise a phase-kick on the state $|11\cdots 1\rangle$ (with no phase induced on any other computational basis state), and then undoing the mapping from $|11\cdots 1\rangle$ to $|u\rangle$. (These operations would be described by the linear transformation $-R_u$, but this distinction is irrelevant to the actual quantum states.) This involves declaring an auxiliary quantum register, which we suppose we can do if n represents a global parameter related to the oracle O_f .

```

define R_u q:
  # perform a reflection through the state  $|u\rangle$  on  $n$  qubits,
  # where we assume that  $n$  is a globally-defined parameter
  # related to the oracle  $O_f$ 
  operand register q:  $n$  qubits

  register a:  $n$  qubits                                # auxiliary workspace
  H q                                                    # map  $|u\rangle$  to  $|00\cdots 0\rangle$ 
  X q                                                    # map  $|00\cdots 0\rangle$  to  $|11\cdots 1\rangle$ 
  Ry(-pi/2) a[n-1]                                       # map  $a[n-1]$  to  $|-\rangle$ 
  MultiCtrlX[n-1] q, a[0:n-2], a[n-1]                 # phase-kick
  Ry(pi/2) a[n-1]                                       # restore  $a[n-1]$  to  $|0\rangle$ 
  X q                                                    # map  $|11\cdots 1\rangle$  to  $|00\cdots 0\rangle$ 
  H q                                                    # map  $|00\cdots 0\rangle$  to  $|u\rangle$ 
  stop

```

also a rotation by 2θ for any other vector in the plane apart from the $|u\rangle$ axis, such as of the vector $|\beta\rangle$ itself.) We call G_f the ‘Grover iterator’.

The fact that G_f realises a rotation of 2θ in the $|a\rangle, |\beta\rangle$ plane suggests the following approach to finding the value of $a \in \{0, 1\}^n$:

1. Let $\tilde{s} = \frac{\pi}{4\theta} - \frac{1}{2}$, so that $(2\tilde{s}+1)\theta = \pi/2$; and let s be the nearest integer to $\tilde{s}$.
2. Prepare an n -qubit register in the state $|u\rangle$.
3. Perform $G_f = R_u R_f$ on q , a total number of s times.
4. Measure q in the computational basis and return the result.

The effect of repeated applications of G_f is illustrated in Figure 5.7.

Taking s as the closest integer to $\tilde{s}$, it follows that $(2s+1)\theta$ will differ from $\pi/2$ by at most $\theta \approx 2^{-n/2}$. In the interest of simplifying the analysis of this procedure, we modify it slightly as follows. Using the approximation $\theta \approx 2^{-n/2}$ which holds with an error of less than 2^{-n} , consider:

$$\tilde{t} := \frac{\pi}{4} \cdot 2^{n/2} - \frac{1}{2} \quad \text{and} \quad t := \left\lfloor \tilde{t} + \frac{1}{2} \right\rfloor = \left\lfloor \frac{\pi}{4} \cdot 2^{n/2} \right\rfloor. \quad (5.10)$$

(Here, for a real number $p \geq 0$, $\lfloor p \rfloor$ is the floor function which extracts the integer part of p . Then, t is the closest integer to $\tilde{t} - \frac{1}{2}$.) We will then have $\tilde{t} \approx \tilde{s}$ to within an error

Figure 5.7: The effect of repeated applications of G_f . As G_f realises a rotation in the $|a\rangle, |\beta\rangle$ plane by 2θ , repeated applications of it on an initial state of $|u\rangle$ will produce state-vectors with a greater and greater angular displacement from $|\beta\rangle$ until it approaches $\pi/2$ – at which point it will be closely aligned with the unknown state $|a\rangle$.

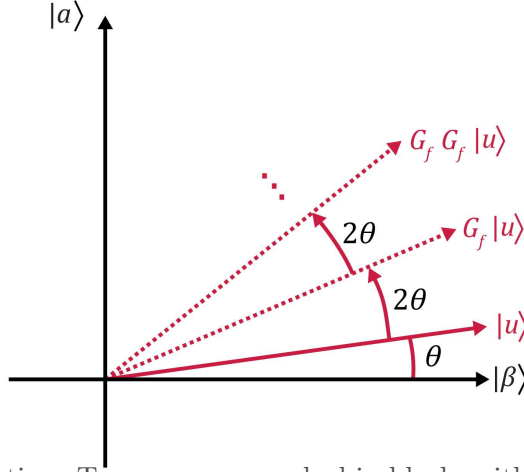


Figure 5.7 Long description: Two axes are marked in black, with $|a\rangle$ on the y-axis and $|\beta\rangle$ the x-axis. The vector $|u\rangle$ is indicated in dark red, pointing right and slightly above the x-axis. The angle this vector makes with the horizontal axis is marked as θ . The vector $G_f |u\rangle$ is indicated in dotted light red, pointing a bit further upwards from $|u\rangle$, and a bit closer to $|a\rangle$. An arc indicates that $G_f |u\rangle$ is rotated in the plane counter-clockwise from $|u\rangle$ by an angle of 2θ . A further vector $G_f G_f |u\rangle$ is also indicated in dotted light red, pointing still further upwards from $|u\rangle$ and still a bit closer to $|a\rangle$. An arc indicates that $G_f G_f |u\rangle$ is rotated in the plane counter-clockwise from $G_f |u\rangle$ by a further angle of 2θ . Ellipses are shown counter-clockwise from the arrow indicating $G_f G_f |u\rangle$, to suggest a continuation of this pattern.

of 2^n , and $(2t+1)\theta$ will differ from $\pi/2$ by at most $\theta \approx 2^{-n/2}$. We then consider the following alternative procedure:

- 1'. Let $\tilde{t} = \frac{\pi}{4} \cdot \sqrt{2^n}$, and let $T = \lfloor \tilde{t} - 1 \rfloor$.
- 2'. Prepare an n -qubit register in the state $|u\rangle$.
- 3'. Perform $R_u R_f$ on q , a total number of t times.
- 4'. Measure q in the computational basis and return the result.

This procedure is illustrated in Figure 5.8 and is known as Grover's algorithm. Note that this involves $\mathcal{O}(2^{n/2})$ applications of G_f and so in particular involves $\mathcal{O}(2^{n/2})$ invocations of the oracle O_f . The number of additional operations is a small polynomial factor more than this: if we count the oracle itself as a single operation, the total number of operations involved is $\mathcal{O}(n 2^{n/2})$. This is an exponential amount of work as n increases, but as we observed before, it is much better than $\mathcal{O}(2^n)$ operations.

The effectiveness of Grover's algorithm is as follows. Let $|\psi\rangle = (G_f)^t |u\rangle$ be the state which is produced at the end of Step 3'. The angle between $|\psi\rangle$ and $|a\rangle$ in the $|a\rangle, |\beta\rangle$

plane is just the amount by which $(2t+1)\theta$ differs from $\pi/2$. We may use the inequality $\cos(\varepsilon) \geq 1 - \frac{1}{2}\varepsilon^2$, which holds for all $\varepsilon \geq 0$ and also yields a good approximation of $\cos(\varepsilon)$ when the value of ε is small. Then we have:

$$|\langle a | \psi \rangle|^2 = \cos\left(\frac{\pi}{2} - (2T+1)\theta\right)^2 > \cos(2^{-n/2})^2 \geq (1 - 2^{-n-1})^2 > 1 - 2^{-n}. \quad (5.11)$$

Then, the probability by which step 4' yields the result $a \in \{0, 1\}^n$ is 1, up to n bits of precision. It will almost never be precisely 1; but the probability of failure can be reduced if desired by repeating it for as many repetitions as you like. (The probability of obtaining any particular outcome $x \neq a$ will be less than $(2^n - 1)^{-1} \cdot 2^{-n}$; so if you obtain the same outcome x from two consecutive trials, you can be very confident that $x = a$ for a large n .)

5.3 Search without uniqueness

Grover's algorithm, as described in the preceding section, assumes that there be exactly one $a \in \{0, 1\}^n$ for which $f(a) = 1$. What happens if this is not the case? The two operations **R.f** and **R.u** are still operations that we can perform, but they do not have precisely the effect that we have described before. Now we consider what happens instead.

For the sake of brevity, we may refer to an $x \in \{0, 1\}^n$ for which $f(x) = 1$, as 'marked'; the other $x \in \{0, 1\}^n$ for which $f(x) = 0$ we call 'unmarked'. We consider the cases of different numbers of marked elements next.

(a) No marked elements

If there are no $x \in \{0, 1\}^n$ at all for which $f(x) = 1$, then **R.f** has no effect at all. Consider the implementation of **R.f** described in Figure 5.2. For each component $|x\rangle$ in the state of the register **q**, the final instruction of **R.f** performs the transformation:

$$|x, -\rangle_{\mathbf{q}, \mathbf{t}} = \frac{1}{\sqrt{2}} |x, 0\rangle_{\mathbf{q}, \mathbf{t}} - \frac{1}{\sqrt{2}} |x, 1\rangle_{\mathbf{q}, \mathbf{t}} \xrightarrow{\mathbf{0.f}} \frac{1}{\sqrt{2}} |x, 0\rangle_{\mathbf{q}, \mathbf{t}} - \frac{1}{\sqrt{2}} |x, 1\rangle_{\mathbf{q}, \mathbf{t}}, \quad (5.12)$$

... which is to say, no transformation at all. The effect of the Grover iterator on the state $|u\rangle$ is then:

$$|u\rangle_{\mathbf{q}} \xrightarrow{\mathbf{R.f}} |u\rangle_{\mathbf{q}} \xrightarrow{\mathbf{R.u}} |u\rangle_{\mathbf{q}}. \quad (5.13)$$

Any number of iterations of G_f applied to $|u\rangle$ has no effect. The final measurement of **GroversAlg** then yields a bit-register with a uniformly random $x \in \{0, 1\}^n$, which of course is guaranteed to satisfy $f(x) = 0$.

This outcome is not too bad: if there is no $x \in \{0, 1\}^n$ such that $f(x) = 1$, then Grover's algorithm produces an outcome which seems to confirm this fact.

Figure 5.8: Grover's algorithm. A subroutine **GroversAlg** to find (with high probability) the string $a \in \{0,1\}^n$ such that $f(a) = 1$, under the promise that a exists and is unique. This subroutine uses the subroutine **GroverIterate**(T) to perform a specifically chosen number of repetitions T of the Grover iterator $G_f = R_u R_f$ on a register q and produces the result in an n -bit register r .

```

define GroverIterate( $T$ )  $q$  :
    # perform the Grover iterator  $T$  times on  $q$ 
    parameter integer  $T$ 
    operand register  $q$ :  $n$  qubits # assumed to be initially  $|u\rangle$ 

    !integer  $t := T$  # a loop counter
    @loop:
        !set  $t := t - 1$  # decrement counter
        !jump_if_zero  $t$ , @done
         $R_f$   $q$  # perform  $G_f$  on
         $R_u$   $q$  # the register  $q$ 
        !jump @loop
    @done:
    stop

define GroversAlg  $r$  :
    # Grover's algorithm, assuming a single element  $a \in \{0,1\}^n$ 
    # such that  $f(a) = 1$ 
    operand register  $r$ :  $n$  bits # to produce the output
    register  $q$ :  $n$  qubits # workspace for the algorithm

    !real  $\tilde{T} := \pi/4 * \sqrt{2^n}$ 
    !integer  $T := \text{floor}(\tilde{T} - 1)$ 

    MxInit  $q$  # initialise  $q$  in state  $|u\rangle = |+\rangle^{\otimes n}$ 
    GroverIterate ( $T$ )  $q$  # do  $T$  repetitions of Grover iterator
    Mz  $q \rightarrow r$  # measure the result and produce as output
    stop

```

(b) Exactly four marked elements

The real difficulty arises from the case in which there is more than one $x \in \{0, 1\}^n$ such that $f(x) = 1$. For instance, suppose that there are exactly four inputs $a_0, a_1, a_2, a_3 \in \{0, 1\}^n$ such that $f(a_1) = f(a_2) = 1$. In this case, we would decompose $|u\rangle$ as:

$$|u\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=1}} |x\rangle + \frac{1}{\sqrt{2^n}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=0}} |x\rangle = \frac{2}{\sqrt{2^n}} |\alpha\rangle + \sqrt{1 - \frac{4}{2^n}} |\beta\rangle, \quad (5.14)$$

where in this case we define the states:

$$|\alpha\rangle = \frac{1}{\sqrt{4}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=1}} |x\rangle = \frac{1}{2} (|a_0\rangle + |a_1\rangle + |a_2\rangle + |a_3\rangle), \quad |\beta\rangle = \frac{1}{\sqrt{2^n - 4}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=0}} |x\rangle. \quad (5.15)$$

Again, recall that we cannot easily prepare $|\alpha\rangle$; we only know that we can define such a state and consider $|u\rangle$ as a linear combination of $|\alpha\rangle$ and $|\beta\rangle$. This generalises the setting of a unique $a \in \{0, 1\}^n$ for which $f(a) = 1$, by taking $|\alpha\rangle$ to be the uniform superposition over all strings $x \in \{0, 1\}^n$ for which $f(x) = 1$. (If there is only one such x , then that ‘uniform superposition’ is just $|a\rangle$.)

We may consider the $|\alpha\rangle, |\beta\rangle$ plane, which contains the state $|u\rangle$; and we may consider the angle θ between $|u\rangle$ and $|\beta\rangle$ as before. In this case, we will have $\sin(\theta) = 2/\sqrt{2^n}$, and in particular $\theta \approx 2/\sqrt{2^n}$ to about n bits of precision. We may then make the following observations:

- The operation R_f represents a reflection in the $|\alpha\rangle, |\beta\rangle$ plane, and in particular still reflects it through the $|\beta\rangle$ axis (where $|\beta\rangle$ remains the superposition of all states $|x\rangle$ for which $f(x) = 0$).
- The reflection R_u remains a reflection through the axis defined by the $|u\rangle$ state – and we may still define a Grover iterator $G_f = R_u R_f$ which realises a rotation of 2θ in the $|\alpha\rangle, |\beta\rangle$ plane.

However – if we perform G_f on $|u\rangle$ some number $t \approx \frac{\pi}{4}\sqrt{2^n} - \frac{1}{2}$ times as we do in Grover’s algorithm, the state that we obtain is one which is in the $|\alpha\rangle, |\beta\rangle$ plane which is at an angle of:

$$(2t + 1)\theta \approx \left(\frac{\pi}{2}\sqrt{2^n}\right) \cdot \frac{2}{\sqrt{2^n}} = \pi, \quad (5.16)$$

accurate to about n bits of precision. That is, $(G_f)^t |u\rangle$ would be closely aligned with the state-vector $-|\beta\rangle$. Then, a measurement in the computational basis would (with high probability) produce some $x \in \{0, 1\}^n$ such that $f(x) = 0$ – despite the fact that there are four times as many ‘marked’ elements as in the case we considered before!

What we see here is that even when there is more than one $x \in \{0, 1\}^n$ for which $f(x) = 1$, the Grover iterator represents some sort of rotation in the $|\alpha\rangle, |\beta\rangle$ plane – and that the angle 2θ of the rotation depends on the number of ‘marked’ elements x , for

which $f(x) = 1$. Because the Grover iterator is a rotation, it does not always increase the amplitudes of the components $|x\rangle$ for which $f(x) = 1$: repeated application can cause the amplitudes of those components to increase and then decrease (and then increase again, etc.) Without knowing the angle of rotation 2θ , we cannot know precisely how many times to apply G_f to obtain a state closely aligned with $|\alpha\rangle$.

(c) Solving the problem with an unknown number of marked elements

Let M be the number of marked elements in $\{0, 1\}^n$. We suppose in the equation that follows that $0 < M < 2^n$. Generalising the analysis of the case mentioned before, we may $|u\rangle$ as:

$$|u\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=1}} |x\rangle + \frac{1}{\sqrt{2^n}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=0}} |x\rangle = \sqrt{\frac{M}{2^n}} |\alpha\rangle + \sqrt{1 - \frac{M}{2^n}} |\beta\rangle, \quad (5.17)$$

where we define the states:

$$|\alpha\rangle = \frac{1}{\sqrt{M}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=1}} |x\rangle, \quad |\beta\rangle = \frac{1}{\sqrt{2^n - M}} \sum_{\substack{x \in \{0,1\}^n \\ f(x)=0}} |x\rangle. \quad (5.18)$$

Again, we consider the $|\alpha\rangle, |\beta\rangle$ plane, which contains the state $|u\rangle$. We also consider the angle θ between $|u\rangle$ and $|\beta\rangle$, which then satisfies $\sin(\theta) = \sqrt{M/2^n}$. Without knowing M in advance, it is impossible to determine θ even approximately. But $G_f = \mathbf{R}_u \mathbf{R}_f$ still realises a rotation of 2θ in the $|\alpha\rangle, |\beta\rangle$ plane, and we know that either $\theta = 0$ or $\theta > 1/\sqrt{2^n}$ (though θ may be very close to $1/\sqrt{2^n}$).

If we **did** know the value of M precisely, we could consider a very specific number of iterations t , such that $(G_f)^t |u\rangle$ was as closely aligned to $|\alpha\rangle$ as possible. This would allow us to produce a random marked element almost with certainty, as measuring $(G_f)^t |u\rangle$ in the computational basis would produce approximately the same distribution of outcomes as measuring $|\alpha\rangle$ in that basis. However, to find **some** marked element $a \in \{0, 1\}^n$, we do not need to produce a state which is very closely aligned with $|\alpha\rangle$. It is enough to produce a state which has a large enough inner product with $|\alpha\rangle$, enough times to make it likely that we would produce a marked element. Consider a state $|\psi\rangle$ of the form:

$$|\psi\rangle = \sin(\phi) |\alpha\rangle + \cos(\phi) |\beta\rangle \quad (5.19)$$

for some angle ϕ . If we perform $\mathbf{0_f}$ on $|\psi, 0\rangle_{\mathbf{q}, \mathbf{p}}$ for some additional qubit $\mathbf{p}$, we obtain:

$$\begin{aligned} |\psi, 0\rangle_{\mathbf{q}, \mathbf{p}} &= \sin(\phi) |\alpha, 0\rangle_{\mathbf{q}, \mathbf{p}} + \cos(\phi) |\beta, 0\rangle_{\mathbf{q}, \mathbf{p}} \\ &\xrightarrow{\mathbf{0_f}} \sin(\phi) |\alpha, 1\rangle_{\mathbf{q}, \mathbf{p}} + \cos(\phi) |\beta, 0\rangle_{\mathbf{q}, \mathbf{p}}. \end{aligned} \quad (5.20)$$

If we then measure $\mathbf{p}$, the probability that we obtain the outcome ‘1’ is $\sin^2(\phi)$. Conditional on that happening, the state of $\mathbf{q}$ in that case would collapse to exactly $|\alpha\rangle$,

Figure 5.9: Schematic representation of states obtained by randomly selecting the number of times to apply the Grover iterator G_f . As we do not know the rotation angle realised by G_f in the $|\alpha\rangle, |\beta\rangle$ plane, we may choose to perform a number of rotations $t = \lfloor \frac{\pi}{4}\sqrt{2^m} - 1 \rfloor$, where we select $m \in \{0, 1, \dots, n\}$ in sequence. Applying this to $|u\rangle$ will realise a range of possible rotations of $|u\rangle$ in the $|\alpha\rangle, |\beta\rangle$ plane. This is represented schematically here, by vectors in the plane pointing in various directions, clustered among small angles from $|\beta\rangle$ but increasing exponentially in their angular spacing. It may be that none of these align very closely with $|\alpha\rangle$, but many of them will have a substantial inner product with $|\alpha\rangle$. With enough of a good many repetitions, this suffices to have a high probability of finding a marked element, if one exists.

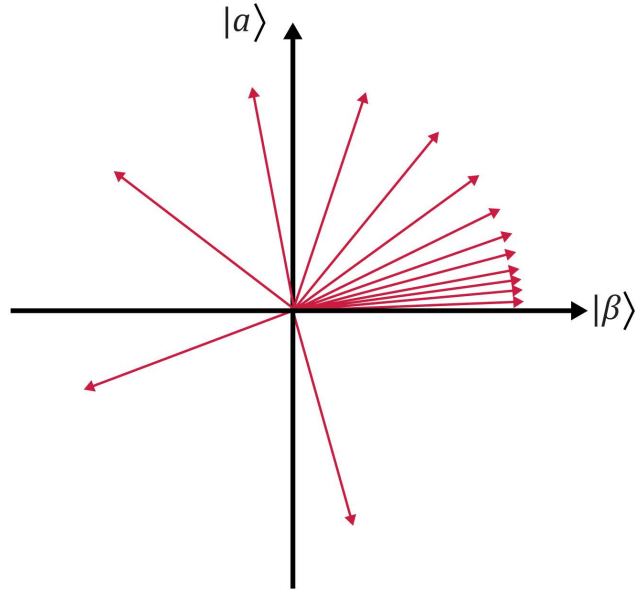


Figure 5.9 Long description: Two axes are marked in black, with $|\alpha\rangle$ on the y-axis and $|\beta\rangle$ the x-axis. Many other vectors are indicated in light red. Several of these are pointing mostly to the right and slightly upwards, at small angular displacements from each other, representing small counter-clockwise rotations. The light red vectors at larger counter-clockwise rotations are increasingly widely spaced from one another. Some of the vectors are at large enough counter-clockwise rotations to be pointing mostly upwards, or upwards and to the left, or even downwards to the left or right.

and measuring $\mathbf{q}$ in the computational basis would yield a random $x \in \{0, 1\}^n$ such that $f(x) = 1$.

So, rather than carefully selecting a number of iterations $t \in \{t_0, t_1, \dots, t_n\}$ to apply the Grover iterator G_f to, we can instead consider a **range** of times that we can apply G_f , realising many different states $|\psi_k\rangle = (G_f)^{t_k} |u\rangle$. (This is illustrated in Figure 5.9.) Our aim in doing so is to produce at least a few states $|\psi_k\rangle$ for which $|\langle\alpha|\psi_k\rangle|^2$ is significant, for example, at least $\frac{1}{2}$. Figure 5.10 demonstrates an implementation of this algorithm where we take the range of repetitions $\{t_0, t_1, \dots, t_n\}$ of G_f , to be given by $t_k = \lfloor \frac{\pi}{4}\sqrt{2^k} - 1 \rfloor$. Taking $\theta \approx 1/\sqrt{2^n}$, is more than enough to ensure that we have a high

probability of measuring a marked element, at least when we take $t = t_n$. For any value of $\theta > 0$, this is also enough to ensure that there are at least two values of t_k for which $\sin^2((2t_k + 1)\theta) \geq \frac{1}{2}$. The probability of finding a marked element is then at least $3/4$, and in general will be significantly higher.

Summary

We have seen how, given an ‘oracle’ subroutine `O_f` to perform a unitary transformation $|x, t\rangle \mapsto |x, t \oplus f(x)\rangle$ for some function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, we can realise a quantum algorithm which solves the ‘search’ problem by using `O_f` in a way which cannot be described sensibly in terms of a classical computation.

Setting aside the complexity of realising a large quantum computer (!), the oracle `O_f` itself is no more difficult to realise as a quantum computation than as a reversible computation: the trick is to use other quantum operations to probe the behaviour of the oracle `O_f` and in so doing find inputs $x \in \{0, 1\}^n$ for which $f(x) = 1$ (provided that such inputs exist). In effect, this approaches the search problem by taking the oracle `O_f` and treating it as a sort dynamical process whose behaviour we can analyse in new and interesting ways that are not possible with conventional computers.

Figure 5.10: An implementation of Grover’s algorithm for an unknown number of marked items. This uses a loop which tries to find a marked item, by applying larger and larger numbers of repetitions of the Grover iterator. The loop is interrupted once an actual marked item is found, which we test using a few additional queries to the oracle `O_f`.

```

define QuantumSearch r, s :
  # Modification of Grover's algorithm, for an unknown number
  # of marked elements, and an operand s explicitly indicating
  # success or failure in finding a marked element
  operand register r: n bits      # to produce the output
  operand bit s                    # bit to indicate 'success'
  register q: n qubits            # workspace for the algorithm
  qubit p                          # to measure if element is marked

  !integer m      := 0              # a loop iterator
  !real tildeT    := 0              # for iteration estimates
  !integer T      := 0              # for number of Grover iterations

  @loop:
    !jump_if_gtr m, n, @done        # jump to @done if m > n
    !set tildeT := pi/4 * sqrt(2^m) # compute a number of
    !set T      := floor(tildeT - 1) # iterations to perform

    MxInit q                        # initialise q in state  $|u\rangle = |+\rangle^{\otimes n}$ 
    MzInit p                        # initialise p in state  $|0\rangle$ 
    GroverIterate (T) q             # do T repetitions of Grover iterator
    O_f q, p                       # coherently test: state in q marked?
    Mz p -> s                      # measure whether element is marked
    Mz q -> r                      # sample a specific bitstring

    NOT s                          # negate success status
    !jump_if_zero s, @done          # if we succeeded, break the loop
    !set m := m+1                   # increase number of iterations
    !jump @loop

  @done:
    NOT s                          # undo negation of the success status
  stop

```